10 — Interpretabilidad del Modelo

Pipeline CAD-TBX11K | Diagnóstico Asistido por Computador en Tuberculosis

Este notebook estudia **qué características usa el modelo final** seleccionado en el notebook 08 (SVM lineal) mediante SHAP. No evalúa rendimiento: las métricas siguen siendo las out-of-fold (OOF) del notebook 09.

Decisión metodológica importante

El `SVC` multiclase usa **one-vs-one (OvO)** internamente. `shap.LinearExplainer` explica los **tres hiperplanos pairwise**, no el score OvR ni probabilidades. El notebook verifica esa correspondencia y etiqueta cada eje como:

Eje SHAP	Margen pairwise	Positivo favorece
0	health vs sick	health
1	health vs tb	health
2	sick vs tb	sick

Por tanto, la evidencia hacia `tb` es `-SHAP[:, :, 1]` y `-SHAP[:, :, 2]`.

Estructura

1. Carga y verificación de artefactos.
2. Preparación del espacio de interpretación y verificación de SHAP.
3. SHAP global por margen pairwise.
4. Interpretación de `tb` frente a `health` y `sick`.
5. Importancia por familia de características.
6. Explicaciones individuales OOF con el modelo del fold correspondiente.
7. Limitaciones y discusión.

0. Configuración del entorno

In [1]:
```
# Se configura el path del proyecto y se importan las librerías necesarias.
# Referencias:
#   SHAP: https://shap.readthedocs.io/en/latest/
#   SVC lineal: https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html

import sys, os, warnings, json
warnings.filterwarnings('ignore')

from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import joblib
import shap

from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.model_selection import StratifiedKFold
```

```
from sklearn.metrics import balanced_accuracy_score, recall_score

PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

from src.selection import build_selector
from src.visualization import set_publication_style

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

CLASS_NAMES = ['health', 'sick', 'tb']
PAIR_LABELS = ['health vs sick', 'health vs tb', 'sick vs tb']

DATA_DIR    = Path('..') / 'data'
MODELS_DIR  = Path('..') / 'models'
REPORTS_DIR = Path('..') / 'reports'
REPORTS_DIR.mkdir(exist_ok=True)

set_publication_style()
print('Entorno configurado correctamente.')
print(f'SHAP {shap.__version__}')
```

```
Entorno configurado correctamente.
SHAP 0.54.1
```

1. Carga y verificación de artefactos

In [2]:

```
# Se comprueban los cuatro artefactos que deben ser compatibles:
#   08_svm.joblib, 08_manifest.json, 06_selected_feature_names.json,
#   08_oof_predicciones.csv (más X_full/y_full, scores y parámetros por fold).

ARCHIVOS = [
    DATA_DIR / 'X_full.npy',
    DATA_DIR / 'y_full.npy',
    DATA_DIR / 'features_tbx11k.csv',
    DATA_DIR / '06_selected_feature_names.json',
    DATA_DIR / '08_oof_predicciones.csv',
    DATA_DIR / '08_oof_scores.npz',
    DATA_DIR / '08_cv_por_fold.csv',
    MODELS_DIR / '08_svm.joblib',
    MODELS_DIR / '08_manifest.json',
]
for archivo in ARCHIVOS:
    if not archivo.exists():
        raise FileNotFoundError(f'No se encontró {archivo}. Ejecuta primero el notebook 08.')

modelo = joblib.load(MODELS_DIR / '08_svm.joblib')
selector = modelo.named_steps['selector']
scaler = modelo.named_steps['scaler']
clf = modelo.named_steps['clf']

print('Pipeline :', [paso[0] for paso in modelo.steps])
print('Selector :', type(selector).__name__, '| N =', selector.top_idx_.shape[0])
print('Escalador:', type(scaler).__name__)
print('Clasificador:', type(clf).__name__)
print('  kernel :', clf.kernel)
print('  C      :', clf.C)
print('  classes:', clf.classes_)
print('  coef_  :', clf.coef_.shape, '| intercept_ :', clf.intercept_.shape)
print('  decision_function_shape:', clf.decision_function_shape)

with open(MODELS_DIR / '08_manifest.json', encoding='utf-8') as f:
    manifest = json.load(f)
print()
print('Modelo seleccionado en 08:', manifest['seleccionado_cv'])
print('Selector en 08:', manifest['selector'])
print('Parámetros SVM:', manifest['mejores_params']['SVM'])

# Correspondencia exacta de las 1000 features.
feat_cols = pd.read_csv(DATA_DIR / 'features_tbx11k.csv', nrows=0).columns.tolist()
```

```
feat_cols = [c for c in feat_cols if c not in ('image_path', 'class_name', 'label')]
pipeline_names = [feat_cols[i] for i in selector.top_idx_]
with open(DATA_DIR / '06_selected_feature_names.json', encoding='utf-8') as f:
    names_06 = json.load(f)

if pipeline_names != names_06:
    raise ValueError('Las features del pipeline no coinciden, en orden, con el notebook 06.')

print()
print('Features: pipeline y 06 coinciden en conjunto y orden (1000).')
print('Primeras 5:', pipeline_names[:5])

X_full = np.load(DATA_DIR / 'X_full.npy')
y_full = np.load(DATA_DIR / 'y_full.npy')
oof = pd.read_csv(DATA_DIR / '08_oof_predicciones.csv')
scores = np.load(DATA_DIR / '08_oof_scores.npz')
fold_params = pd.read_csv(DATA_DIR / '08_cv_por_fold.csv')

if not np.array_equal(oof['y_true'].to_numpy(), y_full):
    raise ValueError('y_true del OOF no coincide con y_full.')
if scores['SVM'].shape != (len(y_full), len(CLASS_NAMES)):
    raise ValueError('Los scores OOF del SVM no tienen la forma esperada.')

print()
print(f'X_full: {X_full.shape} | y_full: {y_full.shape}')
print(f'OOF SVM: BalAcc={balanced_accuracy_score(y_full, oof["SVM"]):.4f} | '
      f'Recall tb={recall_score(y_full, oof["SVM"], labels=[2], average=None)[0]:.4f}')
print('Score SVM OOF: min=%.4f, max=%.4f (funciones de decisión, no probabilidades)'
      % (float(scores['SVM'].min()), float(scores['SVM'].max())))
print()
print('Parámetros SVM por fold:')
print(fold_params[fold_params['Modelo'] == 'SVM'][['Fold', 'Mejores params']].to_string(index=False))
```

```
Pipeline : ['selector', 'scaler', 'clf']
Selector : RFTopNSelector | N = 1000
Escalador: StandardScaler
Clasificador: SVC
  kernel : linear
  C      : 0.003
  classes: [0 1 2]
  coef_  : (3, 1000) | intercept_ : (3,)
  decision_function_shape: ovr

Modelo seleccionado en 08: SVM
Selector en 08: {'metodo': 'RF', 'n_features': 1000}
Parámetros SVM: {'clf__C': 0.003}

Features: pipeline y 06 coinciden en conjunto y orden (1000).
Primeras 5: ['gabor_f0.4_t90_std', 'lbp_1', 'gabor_f0.3_t90_std', 'lbp_25', 'mean']

X_full: (8274, 3300) | y_full: (8274,)
OOF SVM: BalAcc=0.9494 | Recall tb=0.8950
Score SVM OOF: min=-0.3100, max=2.3109 (funciones de decisión, no probabilidades)

Parámetros SVM por fold:
 Fold    Mejores params
    1 {"clf__C": 0.003}
    2 {"clf__C": 0.003}
    3 {"clf__C": 0.003}
    4 {"clf__C": 0.003}
    5 {"clf__C": 0.003}
```

2. Preparación del espacio de interpretación y verificación de SHAP

In [3]:

```
# Se trabaja sobre una muestra estratificada de 500 imágenes por clase para que el
# cálculo sea reproducible y manejable. El modelo explicado es el final (8274 imágenes).
# Referencia: https://shap.readthedocs.io/en/latest/generated/shap.LinearExplainer.html

N_POR_CLASE = 500
rng = np.random.RandomState(RANDOM_STATE)
```

```python
idx = []
for clase in np.unique(y_full):
    idx_clase = np.where(y_full == clase)[0]
    idx.extend(rng.choice(idx_clase, size=N_POR_CLASE, replace=False))
idx = np.array(idx)

X_sample = X_full[idx]
y_sample = y_full[idx]
X_sample_sc = scaler.transform(selector.transform(X_sample))

feature_names = pipeline_names


def grupo_feature(nombre):
    """Asigna cada característica a una familia del extractor."""
    if nombre.startswith('hog_'):
        return 'HOG'
    if nombre.startswith('phog_'):
        return 'PHOG'
    if nombre.startswith('gabor_'):
        return 'Gabor'
    if nombre.startswith('lbp_'):
        return 'LBP'
    if nombre.startswith('glcm_'):
        return 'GLCM'
    if nombre in ('mean', 'std', 'contrast', 'entropy'):
        return 'Intensidad global'
    if nombre[:2] in ('SI', 'SD', 'MI', 'MD', 'II', 'ID'):
        return 'Intensidad regional'
    return 'Otra'


GRUPOS = ['HOG', 'PHOG', 'GLCM', 'LBP', 'Gabor', 'Intensidad global', 'Intensidad regional']
grupos_por_feature = [grupo_feature(n) for n in feature_names]
conteo_grupos = pd.Series(grupos_por_feature).value_counts().reindex(GRUPOS).fillna(0).astype(int)

print('Composición de las 1000 features seleccionadas:')
print(conteo_grupos.to_string())

# Fondo de referencia estratificado para LinearExplainer.
bg_idx = []
for clase in np.unique(y_sample):
    idx_clase = np.where(y_sample == clase)[0]
    bg_idx.extend(rng.choice(idx_clase, size=min(33, len(idx_clase)), replace=False))
background = X_sample_sc[np.array(bg_idx)]

explainer = shap.LinearExplainer(clf, background)
sv = explainer(X_sample_sc)

print()
print(f'SHAP values: {sv.values.shape}  (muestras, features, pares)')
print(f'base_values: {sv.base_values.shape}')
print('Ejes:', dict(zip(range(3), PAIR_LABELS)))

# Verificación: LinearExplainer explica los hiperplanos OvO, no el score OvR.
shap_sum = sv.base_values + sv.values.sum(axis=1)
ovo = (clf.coef_ @ X_sample_sc.T + clf.intercept_[:, None]).T
decision_ovr = clf.decision_function(X_sample_sc)

print()
print('Verificación de la correspondencia:')
print('  max |SHAP - (coef @ x + intercept)| :', float(np.max(np.abs(shap_sum - ovo))))
print('  max |SHAP - decision_function OvR| :', float(np.max(np.abs(shap_sum - decision_ovr))))
if not np.allclose(shap_sum, ovo, atol=1e-5):
    raise ValueError('SHAP no coincide con los hiperplanos OvO; no se puede interpretar como
pairwise.')
print('  -> SHAP explica los márgenes pairwise OvO. No son probabilidades ni el score OvR.')
```

```
Composición de las 1000 features seleccionadas:
HOG                    476
PHOG                   441
GLCM                    10
LBP                     25
Gabor                   32
Intensidad global        3
Intensidad regional     13

SHAP values: (1500, 1000, 3)  (muestras, features, pares)
base_values: (1500, 3)
Ejes: {0: 'health vs sick', 1: 'health vs tb', 2: 'sick vs tb'}

Verificación de la correspondencia:
  max |SHAP - (coef @ x + intercept)| : 1.7133706453620334e-07
  max |SHAP - decision_function OvR| : 8.541120434030237
  -> SHAP explica los márgenes pairwise OvO. No son probabilidades ni el score OvR.
```

3. SHAP global por margen pairwise

In [4]:

```python
# Importancia global: media de |SHAP| sobre las muestras, para cada margen pairwise.
# Se contrasta con los coeficientes del SVM (misma fila de coef_ para cada par).

abs_sv = np.abs(sv.values)  # (n, features, 3)
coef = clf.coef_

fig, axes = plt.subplots(1, 3, figsize=(18, 6))
for par, ax in enumerate(axes):
    imp = abs_sv[:, :, par].mean(axis=0)
    top = np.argsort(imp)[::-1][:15]

    tabla = pd.DataFrame({
        'feature': [feature_names[i] for i in top],
        'mean_abs_shap': imp[top],
        'grupo': [grupo_feature(feature_names[i]) for i in top],
    })
    print()
    print(f'--- {PAIR_LABELS[par]} ---')
    print(tabla.round(6).to_string(index=False))

    # Control con coeficientes del mismo hiperplano.
    top_coef = set(np.argsort(np.abs(coef[par]))[::-1][:20])
    top_shap = set(np.argsort(imp)[::-1][:20])
    print(f'Solapamiento top-20 SHAP / coeficientes: {len(top_shap & top_coef)}/20')

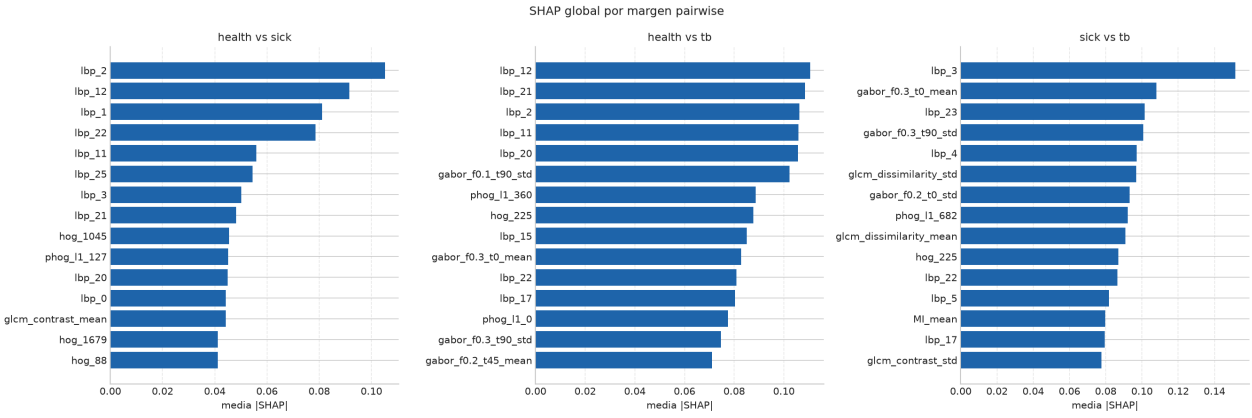
    ax.barh(
        [feature_names[i] for i in top][::-1],
        imp[top][::-1],
        color='#2166AC',
    )
    ax.set_title(PAIR_LABELS[par], fontsize=11)
    ax.set_xlabel('media |SHAP|', fontsize=10)
    ax.grid(True, axis='x', linestyle='--', alpha=0.4)

fig.suptitle('SHAP global por margen pairwise', fontsize=12)
fig.tight_layout()
plt.show()
```

```
--- health vs sick ---
           feature  mean_abs_shap grupo
             lbp_2       0.105230   LBP
            lbp_12       0.091496   LBP
             lbp_1       0.081151   LBP
            lbp_22       0.078551   LBP
            lbp_11       0.055981   LBP
            lbp_25       0.054507   LBP
             lbp_3       0.050167   LBP
            lbp_21       0.048179   LBP
         hog_1045       0.045502   HOG
       phog_l1_127       0.045054  PHOG
            lbp_20       0.044902   LBP
             lbp_0       0.044172   LBP
glcm_contrast_mean       0.044127  GLCM
         hog_1679       0.041156   HOG
           hog_88       0.041145   HOG
Solapamiento top-20 SHAP / coeficientes: 16/20

--- health vs tb ---
           feature  mean_abs_shap grupo
            lbp_12       0.110632   LBP
            lbp_21       0.108575   LBP
             lbp_2       0.106307   LBP
            lbp_11       0.105873   LBP
            lbp_20       0.105739   LBP
 gabor_f0.1_t90_std       0.102348 Gabor
       phog_l1_360       0.088651  PHOG
           hog_225       0.087724   HOG
            lbp_15       0.085087   LBP
 gabor_f0.3_t0_mean       0.082699 Gabor
            lbp_22       0.080811   LBP
            lbp_17       0.080280   LBP
         phog_l1_0       0.077558  PHOG
 gabor_f0.3_t90_std       0.074674 Gabor
gabor_f0.2_t45_mean       0.071124 Gabor
Solapamiento top-20 SHAP / coeficientes: 17/20

--- sick vs tb ---
               feature  mean_abs_shap              grupo
                 lbp_3       0.151578                LBP
     gabor_f0.3_t0_mean       0.107890              Gabor
                lbp_23       0.101417                LBP
     gabor_f0.3_t90_std       0.100773              Gabor
                 lbp_4       0.097007                LBP
 glcm_dissimilarity_std       0.096729               GLCM
      gabor_f0.2_t0_std       0.093112              Gabor
           phog_l1_682       0.092044               PHOG
glcm_dissimilarity_mean       0.090893               GLCM
               hog_225       0.087009                HOG
                lbp_22       0.086336                LBP
                 lbp_5       0.081658                LBP
               MI_mean       0.079747 Intensidad regional
                lbp_17       0.079483                LBP
      glcm_contrast_std       0.077545               GLCM
Solapamiento top-20 SHAP / coeficientes: 18/20
```



4. Interpretación de `tb` frente a `health` y `sick`

In [5]:

```python
# Evidencia hacia cada clase, orientando los márgenes:
#   health: +SHAP par 0 (health vs sick), +SHAP par 1 (health vs tb)
#   sick  : -SHAP par 0 (sick vs health), +SHAP par 2 (sick vs tb)
#   tb    : -SHAP par 1 (tb vs health), -SHAP par 2 (tb vs sick)

EVIDENCIA = {
    'health': [(0, +1), (1, +1)],
    'sick':   [(0, -1), (2, +1)],
    'tb':     [(1, -1), (2, -1)],
}


def importancia_clase(clase):
    """Media de |evidencia| por feature para una clase, sobre sus pares relevantes."""
    valores = np.stack([
        signo * sv.values[:, :, par] for par, signo in EVIDENCIA[clase]
    ], axis=-1)
    return np.abs(valores).mean(axis=(0, 2))


fig, axes = plt.subplots(1, 3, figsize=(18, 6))
for ax, clase in zip(axes, CLASS_NAMES):
    imp = importancia_clase(clase)
    top = np.argsort(imp)[::-1][:15]

    tabla = pd.DataFrame({
        'feature': [feature_names[i] for i in top],
        'mean_abs_evidencia': imp[top],
        'grupo': [grupo_feature(feature_names[i]) for i in top],
    })
    print()
    print(f'--- Evidencia hacia {clase} ---')
    print(tabla.round(6).to_string(index=False))

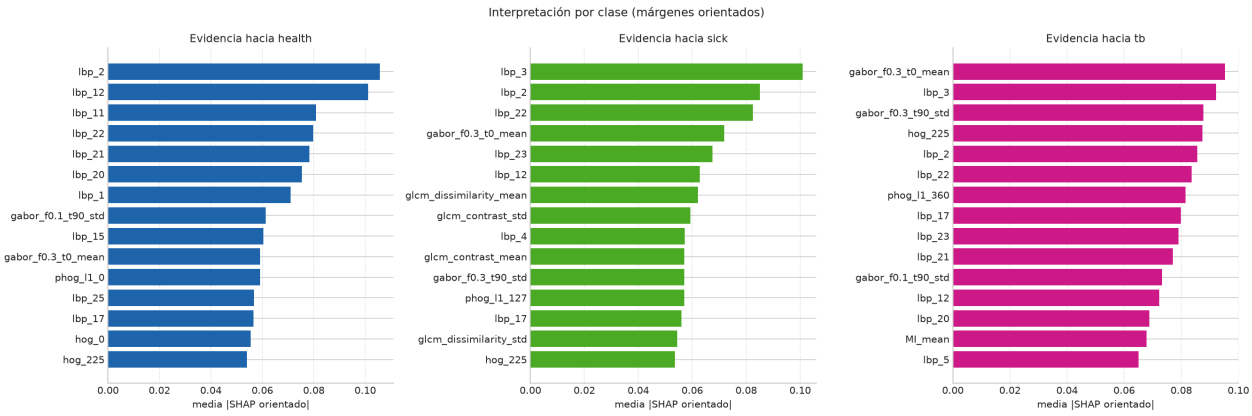
    ax.barh(
        [feature_names[i] for i in top][::-1],
        imp[top][::-1],
        color={'health': '#2166AC', 'sick': '#4DAC26', 'tb': '#D01C8B'}[clase],
    )
    ax.set_title(f'Evidencia hacia {clase}', fontsize=11)
    ax.set_xlabel('media |SHAP orientado|', fontsize=10)
    ax.grid(True, axis='x', linestyle='--', alpha=0.4)

fig.suptitle('Interpretación por clase (márgenes orientados)', fontsize=12)
fig.tight_layout()
plt.show()
```

```
--- Evidencia hacia health ---
           feature  mean_abs_evidencia grupo
             lbp_2            0.105768   LBP
            lbp_12            0.101064   LBP
            lbp_11            0.080927   LBP
            lbp_22            0.079681   LBP
            lbp_21            0.078377   LBP
            lbp_20            0.075321   LBP
             lbp_1            0.071032   LBP
gabor_f0.1_t90_std            0.061241 Gabor
            lbp_15            0.060410   LBP
gabor_f0.3_t0_mean            0.059201 Gabor
         phog_l1_0            0.059024  PHOG
            lbp_25            0.056778   LBP
            lbp_17            0.056474   LBP
             hog_0            0.055422   HOG
           hog_225            0.053913   HOG

--- Evidencia hacia sick ---
               feature  mean_abs_evidencia grupo
                 lbp_3            0.100872   LBP
                 lbp_2            0.085022   LBP
                lbp_22            0.082444   LBP
    gabor_f0.3_t0_mean            0.071796 Gabor
                lbp_23            0.067434   LBP
                lbp_12            0.062745   LBP
glcm_dissimilarity_mean           0.062083  GLCM
     glcm_contrast_std            0.059245  GLCM
                 lbp_4            0.057269   LBP
    glcm_contrast_mean            0.057080  GLCM
    gabor_f0.3_t90_std            0.057067 Gabor
           phog_l1_127            0.057031  PHOG
                lbp_17            0.056075   LBP
 glcm_dissimilarity_std           0.054461  GLCM
               hog_225            0.053555   HOG

--- Evidencia hacia tb ---
           feature  mean_abs_evidencia              grupo
gabor_f0.3_t0_mean            0.095295              Gabor
             lbp_3            0.092219                LBP
gabor_f0.3_t90_std            0.087724              Gabor
           hog_225            0.087366                HOG
             lbp_2            0.085561                LBP
            lbp_22            0.083573                LBP
       phog_l1_360            0.081468               PHOG
            lbp_17            0.079882                LBP
            lbp_23            0.079080                LBP
            lbp_21            0.076985                LBP
gabor_f0.1_t90_std            0.073278              Gabor
            lbp_12            0.072313                LBP
            lbp_20            0.068871                LBP
           MI_mean            0.067901 Intensidad regional
             lbp_5            0.065020                LBP
```



5. Importancia por familia de características

In [6]:

```python
# Se agrega la importancia por familia. Se reporta la media por feature (justa entre
# familias con distinto número de columnas) y el total por familia.

def importancia_grupo(imp_vec):
    filas = []
    for grupo in GRUPOS:
        idx_grupo = [i for i, g in enumerate(grupos_por_feature) if g == grupo]
        if not idx_grupo:
            continue
        filas.append({
            'Grupo': grupo,
            'n_features': len(idx_grupo),
            '%_features': 100 * len(idx_grupo) / len(feature_names),
            'media_por_feature': float(np.mean(imp_vec[idx_grupo])),
            'total_grupo': float(np.sum(imp_vec[idx_grupo])),
        })
    return pd.DataFrame(filas)

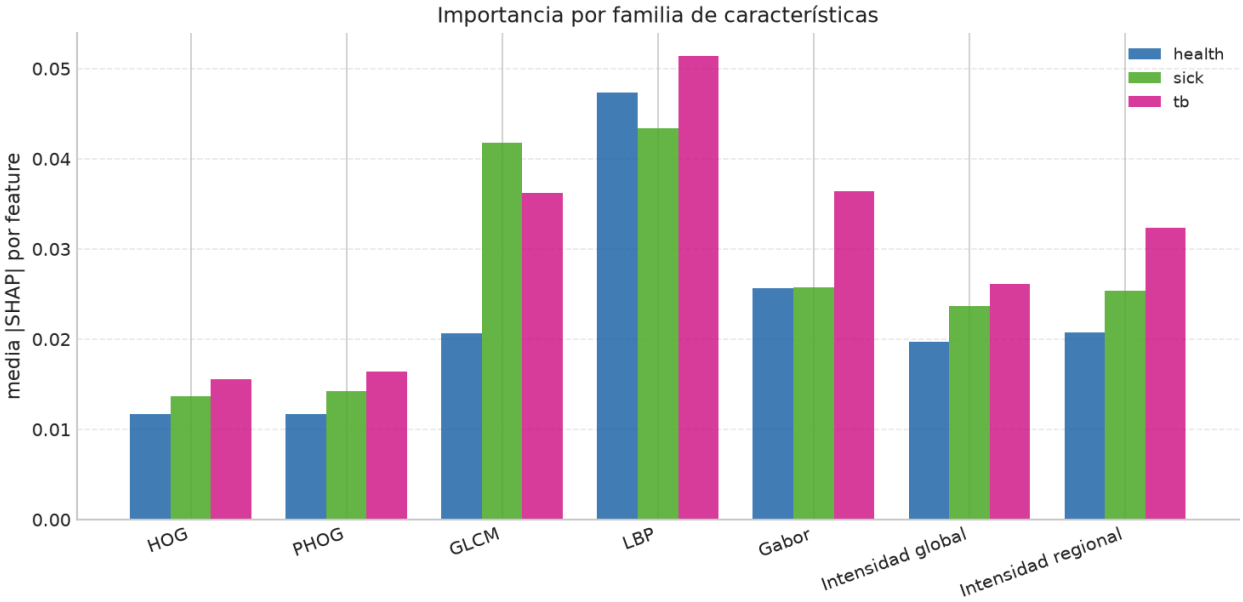

tablas = []
for clase in CLASS_NAMES:
    df_clase = importancia_grupo(importancia_clase(clase))
    df_clase['Clase'] = clase
    tablas.append(df_clase)

tabla_grupos = pd.concat(tablas, ignore_index=True)
print('Importancia media por feature y familia:')
print(tabla_grupos.pivot(index='Grupo', columns='Clase', values='media_por_feature')
      .reindex(GRUPOS).round(6).to_string())

# Gráfico de barras agrupadas (media por feature).
pivote = tabla_grupos.pivot(index='Grupo', columns='Clase', values='media_por_feature').reindex(GRUPOS)
x = np.arange(len(pivote.index))
ancho = 0.26
fig, ax = plt.subplots(figsize=(10, 5))
for k, clase in enumerate(CLASS_NAMES):
    ax.bar(x + (k - 1) * ancho, pivote[clase], ancho,
           label=clase,
           color={'health': '#2166AC', 'sick': '#4DAC26', 'tb': '#D01C8B'}[clase],
           alpha=0.85)
ax.set_xticks(x)
ax.set_xticklabels(pivote.index, rotation=20, ha='right')
ax.set_ylabel('media |SHAP| por feature', fontsize=11)
ax.set_title('Importancia por familia de características', fontsize=12)
ax.legend(fontsize=9)
ax.grid(True, axis='y', linestyle='--', alpha=0.4)
fig.tight_layout()
plt.show()

print()
print('Nota: el total por familia favorece a HOG/PHOG por tener más columnas;')
print('la comparación justa entre familias es la media por feature.')
```

```
Importancia media por feature y familia:
Clase                 health      sick        tb
Grupo
HOG                 0.011685  0.013684  0.015579
PHOG                0.011703  0.014279  0.016423
GLCM                0.020637  0.041737  0.036186
LBP                 0.047317  0.043406  0.051379
Gabor               0.025634  0.025710  0.036389
Intensidad global   0.019687  0.023699  0.026150
Intensidad regional 0.020737  0.025415  0.032314
```




```
Nota: el total por familia favorece a HOG/PHOG por tener más columnas;
la comparación justa entre familias es la media por feature.
```

6. Explicaciones individuales OOF

In [7]:

```python
# Para explicar estrictamente un error OOF se reconstruye el modelo del fold que
# generó esa predicción: mismo split, mismo selector y mismos parámetros guardados.
# El modelo final (8274) se usa para el análisis global, no para estas explicaciones.

cv_outer = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
folds = list(cv_outer.split(X_full, y_full))
fold_of_index = {}
for fold, (tr_idx, va_idx) in enumerate(folds, start=1):
    for i in va_idx:
        fold_of_index[int(i)] = fold

params_por_fold = {
    int(fila['Fold']): json.loads(fila['Mejores params'])
    for _, fila in fold_params[fold_params['Modelo'] == 'SVM'].iterrows()
}

fold_cache = {}


def obtener_modelo_fold(fold):
    """Reconstruye el modelo SVM del fold (selector + escalador + SVC)."""
    if fold in fold_cache:
        return fold_cache[fold]

    tr_idx, va_idx = folds[fold - 1]
    params = params_por_fold[fold]
    C = params.get('clf__C', 0.003)

    selector_f = build_selector('RF', 1000, random_state=RANDOM_STATE, n_jobs=-1)
    selector_f.fit(X_full[tr_idx], y_full[tr_idx])
    X_tr = selector_f.transform(X_full[tr_idx])
    scaler_f = StandardScaler().fit(X_tr)
    X_tr_sc = scaler_f.transform(X_tr)

    clf_f = SVC(
        kernel='linear', class_weight='balanced', C=C,
        cache_size=500, decision_function_shape='ovr',
        random_state=RANDOM_STATE,
    )
    clf_f.fit(X_tr_sc, y_full[tr_idx])

    fold_cache[fold] = (selector_f, scaler_f, clf_f, X_tr_sc, va_idx)
    return fold_cache[fold]
```

```python
y_true = oof['y_true'].to_numpy()
pred_svm = oof['SVM'].to_numpy()

casos = {
    'tb correcto': np.where((y_true == 2) & (pred_svm == 2))[0],
    'tb falso negativo': np.where((y_true == 2) & (pred_svm != 2))[0],
    'tb falso positivo': np.where((y_true != 2) & (pred_svm == 2))[0],
}

seleccion = {nombre: int(rng.choice(indices)) for nombre, indices in casos.items()}

for etiqueta, idx_ejemplo in seleccion.items():
    fold = fold_of_index[idx_ejemplo]
    selector_f, scaler_f, clf_f, X_tr_sc, va_idx = obtener_modelo_fold(fold)

    x_raw_sel = selector_f.transform(X_full[[idx_ejemplo]])
    x_sc = scaler_f.transform(x_raw_sel)
    pred_fold = int(clf_f.predict(x_sc)[0])

    bg_fold = X_tr_sc[rng.choice(len(X_tr_sc), size=min(100, len(X_tr_sc)), replace=False)]
    explainer_f = shap.LinearExplainer(clf_f, bg_fold)
    sv_i = explainer_f(x_sc)

    nombres_fold = [feat_cols[i] for i in selector_f.top_idx_]
    clase_pred = CLASS_NAMES[pred_fold]

    print('=' * 72)
    print(f'{etiqueta} | índice={idx_ejemplo} | fold={fold}')
    print(f'  real={CLASS_NAMES[int(y_true[idx_ejemplo])]} | '
          f'OOF={CLASS_NAMES[int(pred_svm[idx_ejemplo])]} | fold={clase_pred}')
    if pred_fold != int(pred_svm[idx_ejemplo]):
        print('  ADVERTENCIA: la predicción reconstruida no coincide con la OOF.')

    pares = EVIDENCIA[clase_pred]
    for par, signo in pares:
        contrib = signo * sv_i.values[0, :, par]
        top = np.argsort(np.abs(contrib))[::-1][:8]

        tabla = pd.DataFrame({
            'feature': [nombres_fold[i] for i in top],
            'shap_orientado': contrib[top],
            'valor_feature': x_raw_sel[0, top],
        })
        print(f'  Margen {PAIR_LABELS[par]} (orientado a {clase_pred}):')
        print(tabla.round(4).to_string(index=False))
```

```
================================================================================
tb correcto | índice=7960 | fold=5
  real=tb | OOF=tb | fold=tb
  Margen health vs tb (orientado a tb):
           feature  shap_orientado  valor_feature
             lbp_2          0.1980         0.0126
          hog_1520         -0.1830         0.4675
       phog_l1_360          0.1757         0.2217
           hog_225          0.1629         0.2120
gabor_f0.3_t0_mean          0.1146         0.6678
            lbp_1          0.1053         0.0098
            hog_21          0.1025         0.0472
          hog_1605         -0.0813         0.0272
  Margen sick vs tb (orientado a tb):
           feature  shap_orientado  valor_feature
           hog_225          0.1958       0.212000
 gabor_f0.4_t0_std          0.1818       0.947400
       phog_l1_360          0.1677       0.221700
gabor_f0.3_t0_mean          0.1447       0.667800
 gabor_f0.3_t0_std          0.1407       0.929400
            II_std         -0.1404      47.878899
            lbp_4          0.1317       0.011600
         phog_l1_7          0.1016       0.087000
================================================================================
tb falso negativo | índice=8242 | fold=2
  real=tb | OOF=health | fold=health
  Margen health vs sick (orientado a health):
    feature  shap_orientado  valor_feature
    hog_227         -0.1070         0.2302
  phog_l1_0         -0.0995         0.2755
      hog_0         -0.0970         0.2735
phog_l1_214          0.0829         0.2869
     lbp_22         -0.0655         0.0072
     lbp_21         -0.0594         0.0098
     lbp_20         -0.0558         0.0120
     SD_std         -0.0553        55.2883
  Margen health vs tb (orientado a health):
     feature  shap_orientado  valor_feature
   phog_l1_0         -0.1428         0.2755
       hog_0         -0.1281         0.2735
phog_l1_1008         -0.1177         0.0156
      lbp_21         -0.1137         0.0098
      lbp_20         -0.1120         0.0120
     hog_225         -0.0958         0.1411
 phog_l1_360         -0.0916         0.1409
 phog_l1_990         -0.0895         0.0422
================================================================================
tb falso positivo | índice=5491 | fold=5
  real=sick | OOF=tb | fold=tb
  Margen health vs tb (orientado a tb):
           feature  shap_orientado  valor_feature
            lbp_21         -0.2376         0.0060
            lbp_20         -0.1896         0.0076
gabor_f0.3_t90_std          0.1343         1.0424
           hog_224         -0.1281         0.0767
      phog_l1_1147          0.1165         0.2679
          hog_1593          0.1100         0.1242
        phog_l1_83         -0.0998         0.1000
      phog_l1_1149          0.0995         0.2245
  Margen sick vs tb (orientado a tb):
           feature  shap_orientado  valor_feature
             lbp_4          0.1839         0.0106
       phog_l1_785         -0.1682         0.2563
gabor_f0.3_t90_std          0.1634         1.0424
           hog_984         -0.1483         0.1287
             lbp_5          0.1463         0.0124
        phog_l0_77          0.1355         0.0444
          hog_1559         -0.1341         0.0412
        phog_l0_76          0.1340         0.1896
```

7. Limitaciones y discusión

- **SHAP explica los márgenes pairwise OvO**, no el score OvR ni probabilidades. La verificación de la sección 2 muestra que la suma de valores SHAP coincide con `coef_ @ x + intercept_` y no con `decision_function`.
- **Coeficientes y SHAP son complementarios**, no validaciones independientes. El solapamiento top-20 se reporta como control de coherencia.
- **Features correlacionadas**: HOG y PHOG comparten información; SHAP reparte la atribución entre columnas correlacionadas y no debe interpretarse como causalidad.
- **Intensidad y contraste**: si dominan, conviene investigar una posible dependencia del preprocesamiento (CLAHE, saturación de contraste). Por sí solo no demuestra un patrón espurio.
- **Modelo final vs modelos OOF**: el análisis global describe el modelo entrenado con las 8274 imágenes; las explicaciones individuales usan el modelo del fold que produjo cada predicción OOF.
- **Sin test externo ni IDs de paciente**: la estimación de rendimiento sigue siendo la CV OOF del notebook 09, no una validación externa.

Conclusiones

El modelo final es un SVM lineal y SHAP explica sus **tres márgenes pairwise OvO**, no probabilidades ni el score OvR. La verificación numérica de la sección 2 confirma esa correspondencia.

Las 1000 features seleccionadas están dominadas por HOG (476) y PHOG (441), pero la importancia media por feature es mayor en LBP y, para TB, también en Gabor y GLCM:

- **health vs sick**: LBP domina (`lbp_2` , `lbp_12` , `lbp_1` , `lbp_22`); después aparecen HOG/PHOG y GLCM.
- **health vs tb**: LBP + Gabor + PHOG/HOG; aparecen `gabor_f0.1_t90_std` y `gabor_f0.3_t0_mean` .
- **sick vs tb**: LBP + Gabor + GLCM (`glcm_dissimilarity_std` , `glcm_contrast_std`).

En la evidencia orientada a TB los primeros lugares son Gabor (`gabor_f0.3_t0_mean` , `gabor_f0.3_t90_std`), varias columnas LBP, HOG/PHOG y `MI_mean` (intensidad regional). La media por familia confirma que LBP es la familia con mayor importancia por feature; Gabor es la segunda para TB; GLCM pesa más en sick; HOG/PHOG aportan muchas columnas pero con menor importancia individual.

El solapamiento entre las features dominantes de SHAP y los coeficientes del SVM es alto (16/20, 17/20 y 18/20 para los tres márgenes), lo que respalda la coherencia de la interpretación.

En los ejemplos individuales, el TB correcto se apoya en LBP/HOG/PHOG/Gabor; el falso negativo presenta evidencia hacia health en HOG/PHOG; el falso positivo (real sick, predicho tb) se apoya en LBP/Gabor y en contra de TB en algunas columnas LBP. Son explicaciones del modelo del fold correspondiente, no del modelo final.

No se observa un dominio global de intensidad/contraste, aunque `MI_mean` aparece en la evidencia hacia TB y conviene vigilarlo. La interpretación es asociativa: SHAP no demuestra

causalidad y las features correlacionadas comparten atribución.

Fin del pipeline de modelado, evaluación e interpretabilidad.